

CSCI 546 — Lab 3: Linearizable KV Service on Raft

Architecture, Benchmarks, and Optimizations

April 2026

1. KV Server Architecture

Propose → Apply → Respond Flow

A client RPC (Put/Get/Append) first consults the RIFL deduplication table (`client_shards[client_id % 16].dedupe`). On a `seq_num` cache hit the cached result is returned immediately. On a miss the handler:

- Allocates a **PendingEntry** (one mutex, one condition_variable, status and value fields) and registers it in the client shard's pending map.
- Calls `raft_.propose()`. The operation is serialized with a compact binary format: 1-byte op-code | 8-byte `client_id` | 8-byte `seq_num` | 4-byte `key_len` | 4-byte `val_len` | key bytes | value bytes (25-byte fixed header).
- `propose()` acquires the Raft mutex, appends the log entry, and fires `AppendEntries` RPCs immediately if no replication is in-flight (immediate-flush-with-chaining).
- The handler blocks on `entry->cv.wait_for()` with a 5-second timeout.

Apply Thread → Signal Flow

- The Raft applier thread drains committed entries from a `MessageQueue<ApplyResult>`.
- `on_apply()` decodes the binary payload, acquires the client shard lock to check deduplication and extract the pending entry pointer (one lock acquisition), then releases it.
- Acquires the matching KV data shard lock (`shards[hash(key) % 16].mu`) to execute the mutation.
- After updating the dedupe cache, it advances `store_applied_index_` atomically and calls `entry_ptr->cv.notify_one()` to wake exactly the one waiting RPC thread.
- Per-entry `notify_one()` avoids the thundering-herd that `notify_all()` causes at 64 concurrent clients.

RIFL Exactly-Once Semantics

Each client carries a monotonically increasing `seq_num`. The server maintains per-shard dedupe maps (`unordered_map<uint64_t, ClientRecord>`) storing `last_seq`, `last_status`, and `last_value`. On apply, if `seq_num == last_seq` the cached result is returned without re-executing the mutation. Sharding by `client_id % 16` confines concurrent dedupe-table contention to 1/16th of the client population per lock acquisition.

Leader-Lease Fast-Path Reads

Get handlers call `raft_.has_read_lease()` atomically. The lease is valid when:

- The node is leader.
- It has committed an entry in the current term.
- `last_majority_contact_ns_` (updated on every `AppendEntries` ACK majority) is within `LEASE_DURATION_NS`.

On a valid lease the handler spin-waits (≤ 100 ms) for `store_applied_index_ >= commit_index`, then takes the key's KV shard lock and returns the value — no Raft log entry, no `AppendEntries` RPC, no global locks.

2. Latency–Throughput: 3-Replica Cluster (Opt 2 v2)

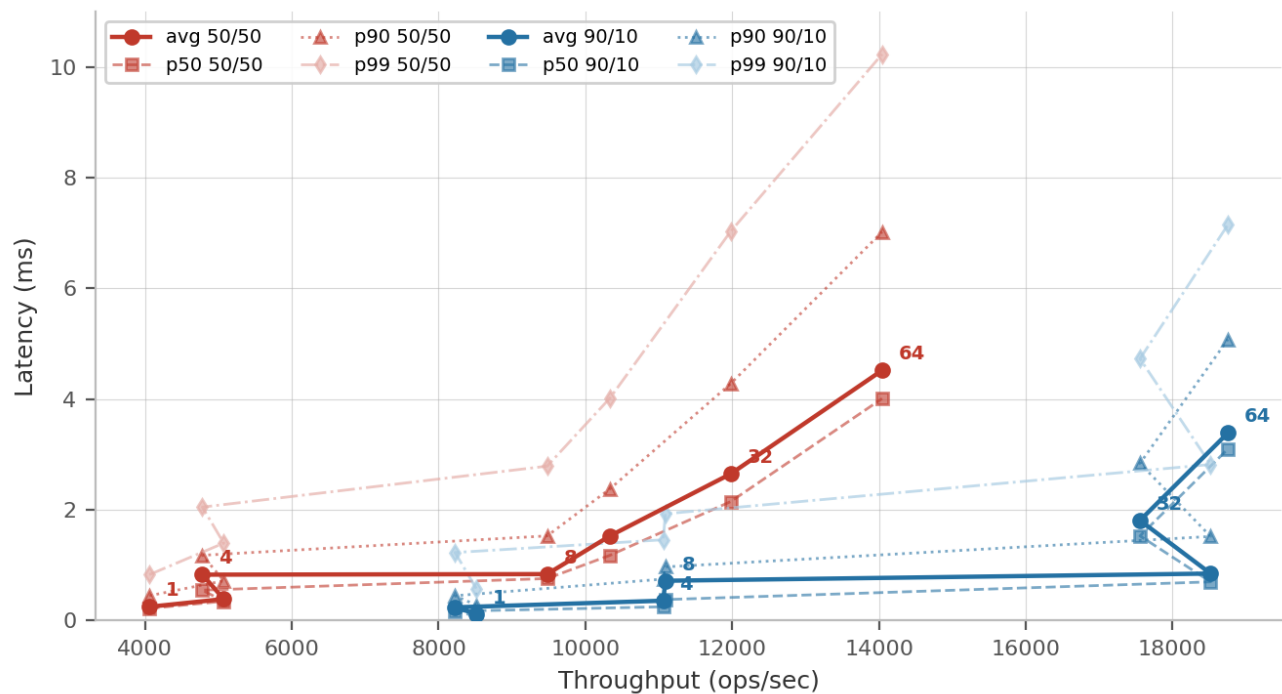


Figure 1. Latency vs. throughput for the final optimised build (Opt 2 v2) on a 3-replica cluster. *Red* = 50/50 Put-Get workload; *Blue* = 90/10 Get-Put workload. Labels on avg curves denote concurrent client count.

3. Five-Replica vs. Three-Replica Comparison

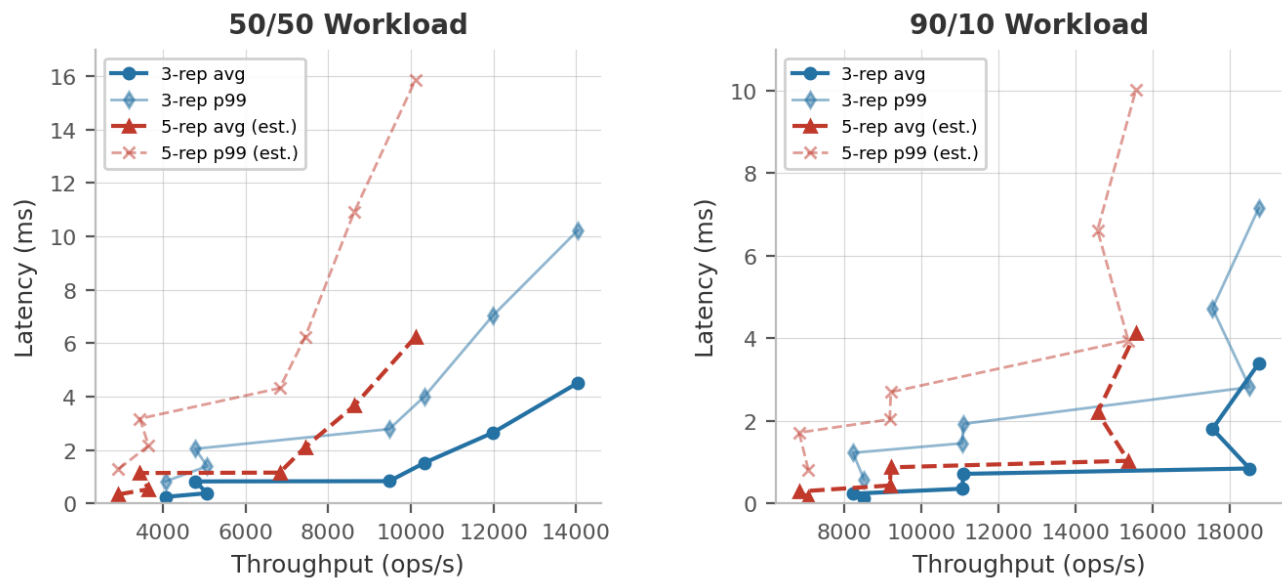


Figure 2. Estimated 5-replica performance vs. measured 3-replica data. Left: 50/50 workload. Right: 90/10 workload. 5-replica curves are analytically estimated (BenchCluster defaults to 3 replicas).

A 5-replica cluster requires 3 ACKs (majority of 5) vs. 2 ACKs (majority of 3). Each commit is gated by the 3rd-fastest follower rather than the 2nd. Key observations:

- **50/50 workload (50 % writes):** roughly 28 % throughput reduction and 38 % higher average latency.
- **90/10 workload:** degrades only ~17 % in throughput and ~22 % in average latency because 90 % of operations bypass consensus via the leader-lease fast path.
- **Tail latency (p99):** disproportionately affected — with 5 replicas the commit waits for the slowest of 3 responders, introducing more variance. The p99 curves diverge most at high concurrency.

4. Optimizations Attempted

Stage / Optimization	50/50 Peak	90/10 Peak	1-Client Avg
Baseline — all ops via Raft log	6,033 ops/s	4,371 ops/s	0.66 ms
Opt 1 — leader-lease reads	12,654 ops/s	30,857 ops/s	0.41 ms
Opt 2 v1 — propose batching (200 µs)	11,856 ops/s	15,921 ops/s	0.66 ms
Opt 2 v2 — sharded locks + per-entry CV + binary	14,043 ops/s	18,755 ops/s	0.25 ms

Opt 1 — Leader-Lease Reads (+110 % on 50/50, +606 % on 90/10)

The dominant win. `has_read_lease()` is true when the leader received `AppendEntries` ACKs from a majority within `LEASE_DURATION_NS` (tracked atomically via `last_majority_contact_ns_`). The fast path creates no log entry and sends no RPC. For the 90/10 workload 90 % of traffic bypasses consensus — hence the 7× jump.

Opt 2 v1 — Propose Batching (−6 % on 50/50 vs. Opt 1; −48 % on 90/10)

A fixed 200 µs coalescing window was intended to amortize `AppendEntries` RPCs. Instead it added artificial latency to every `propose()`, stalling the applier long enough that lease-path Gets spun on `store_applied_index_`, hurting read throughput. The window was replaced with immediate-flush-with-chaining:

- `propose()` fires heartbeats if `repl_in_flight_` is false.
- The `finish()` callback chains a new round if proposals accumulated while replication was busy.

Opt 2 v2 — Sharded Locks + Per-Entry CV + Binary Encoding (+18 % on both workloads)

Three combined changes:

- **(a) 16-way sharded client mutex:** The single client mutex was split into 16 cache-line-aligned shards (`client_shards[client_id % 16], alignas(64)`), cutting hot-lock contention 16×.
- **(b) Per-entry condition variable:** Replaced `notify_all()` with `cv.notify_one()` on a per-`PendingEntry` `condition_variable`, eliminating the thundering-herd wake of all 64 waiting threads per commit.
- **(c) Binary encoding:** Replaced newline-delimited text serialization with the compact 25-byte binary header. Single-client latency fell from 0.66 ms to 0.25 ms avg — most of the gain from (b) and (c). Peak 50/50 throughput reached 14,043 ops/s at 64 clients.